



Universidad
Rafael Landívar
Identidad Jesuita en Guatemala

Facultad de Ingeniería
Facultad de Informática y sistemas
Manejo e Implementación de Archivos
Viernes, 02 de octubre de 2026

Universidad Rafael Landívar
Facultad de ingeniería
Manejo e Implementación de archivos
Sección: 02
Ing. Luis Enrique Rojas



LABORATORIO 13
SECRETS & AUTHENTICATION API

Shayla Mayré De León Monterroso
Carné: 1342925

Guatemala de la Asunción, 02 de octubre de 2026



Nombre: Shayla Mayré De León Monterroso Carné: 1342925

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

a) Abra Postman y cree una nueva colección llamada: MIA – Semana 13.

b) Cree una solicitud llamada: Login.

c) Seleccione el método POST.

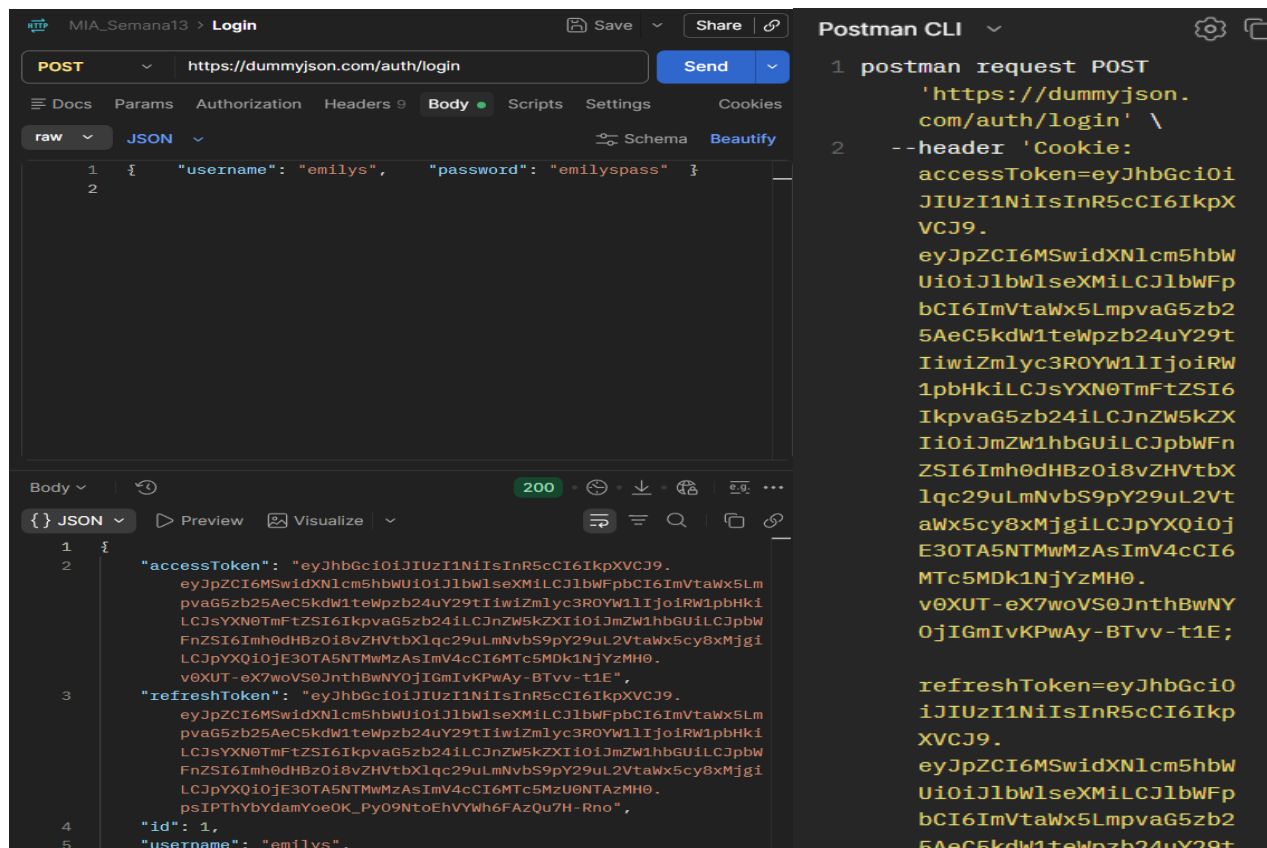
d) Utilice el endpoint: <https://dummyjson.com/auth/login>

e) En Body seleccione raw y JSON.

Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.

```
{ "username": "emilys", "password": "emilyspass" }
```

f) Ejecute la solicitud y localice en la respuesta el campo accessToken.



2. Utilizar token

a) Cree una segunda solicitud llamada: Usuario autenticado.



c) Utilice: <https://dummyjson.com/user/me>

d) En Authorization seleccione Bearer Token.

e) Copie el accessToken obtenido en la solicitud Login.

f) Ejecute la solicitud.

g) Verifique que la API devuelve la información del usuario autenticado.

The screenshot shows a REST client interface with the following details:

- URL:** `https://dummyjson.com/user/me`
- Method:** GET
- Authorization:** Bearer Token
- Token:** A masked token is displayed in the input field.
- Status:** 200
- Body:** JSON response showing user details.

```
1 {
2   "id": 1,
3   "firstName": "Emily",
4   "lastName": "Johnson",
5   "maidenName": "Smith",
6   "age": 29,
7   "gender": "female",
8   "email": "emily.johnson@x.dummyjson.com",
9   "phone": "+81 965-431-3024",
10  "username": "emilys",
11  "password": "emilyspass",
12  "birthDate": "1996-5-30",
13  "image": "https://dummyjson.com/icon/emilys/128",
14  "bloodGroup": "O-",
15  "height": 193.24,
16  "weight": 63.16,
17  "eyeColor": "Green",
```

3. Comprobar qué ocurre sin autenticación

a) Duplique la solicitud Usuario autenticado.

b) Quite el token.

c) Ejecute nuevamente la solicitud.

d) Compare la respuesta con la solicitud autenticada.

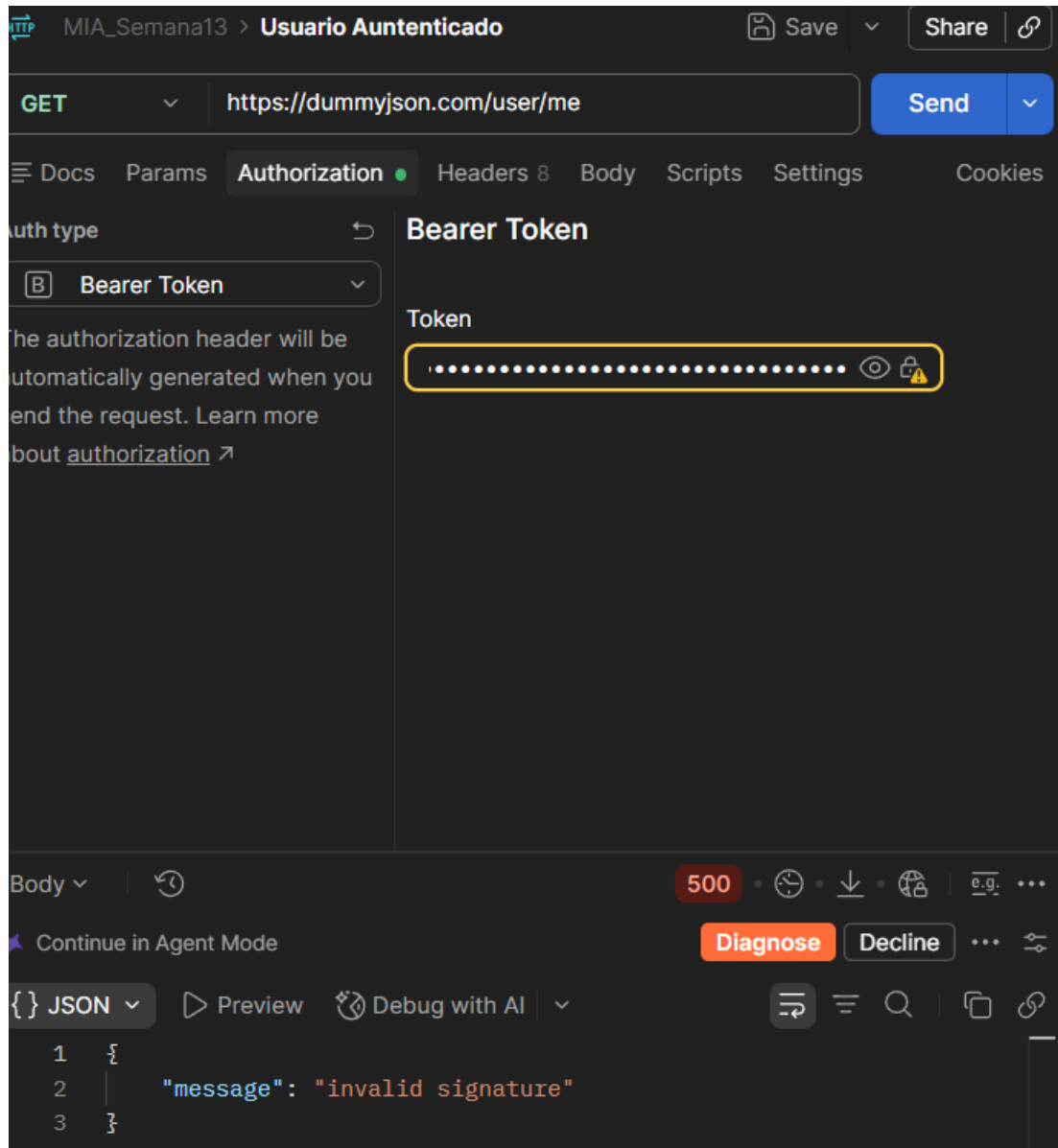
e) Registre el código HTTP y explique la diferencia.



R//Cuando el Bearer Token estaba completo y correcto, la solicitud respondió con código HTTP 200 y mostró la información del usuario autenticado. Al modificar algunos caracteres del token, la solicitud respondió con código HTTP 500 y ya no devolvió la información del usuario, debido a que el token dejó de ser válido para la autenticación.

El Bearer Token funciona como una credencial temporal que permite comprobar que el usuario fue autenticado y autoriza el acceso a recursos protegidos de la API.

Elimine valores del token.



4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.



Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada:

AZURE_STORAGE_CONNECTION_STRING

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

```
string? connectionString =  
    Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING")  
  
if (string.IsNullOrEmpty(connectionString))  
{  
    Console.WriteLine("Error: No se encontro la Connection String.");  
    return;  
}  
  
string containerName = "mia-archivos";
```

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.



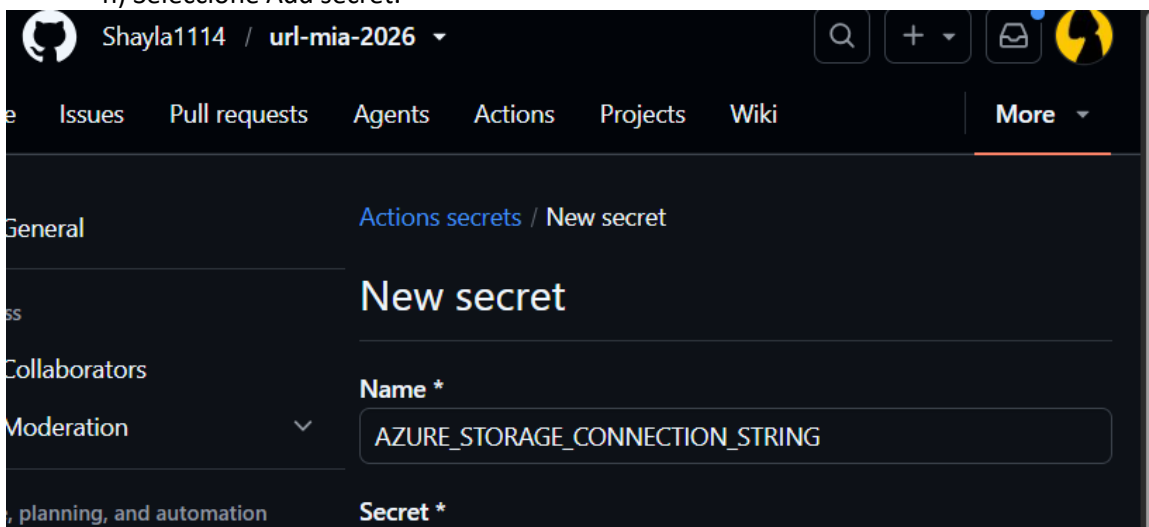
```
Program.cs
1 using Azure.Storage.Blobs;
2 using DotNetEnv;
3
4 Env.Load();
5
6 string? connectionString =
7     Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
8
9 if (string.IsNullOrEmpty(connectionString))
10 {
11     Console.WriteLine("Error: No se encontro la Connection String.");
12     return;
13 }
14
15 string containerName = "mia-archivos";
16
17 BlobServiceClient blobServiceClient =
18     new BlobServiceClient(connectionString);
19
20 BlobContainerClient containerClient =
21     blobServiceClient.GetBlobContainerClient(containerName);
22
23 Console.WriteLine("Conexion con Azure Blob Storage realizada correctamente.");
```

```
log : Se ha restaurado C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-12\lab2\MIA_AzureBlob\MIA_AzureBlob.csproj (en 5.09 s).
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-12\lab2\MIA_AzureBlob> dotnet run
Conexion con Azure Blob Storage realizada correctamente.

=====
MIA - AZURE BLOB STORAGE
=====
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
=====
Seleccione una opcion: 5
Saliendo del programa...
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-12\lab2\MIA_AzureBlob>
```

8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.





Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets

Variables

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets

[New repository secret](#)

Name 

Last updated



AZURE_STORAGE_CONNECTI...

now



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.